

Relational schema design

COMPLETE LEARNING GUIDE

What You'll Learn

A relational schema is the single hardest thing to change once an application is in production with real data in it — unlike application code, which can be refactored freely, a schema change on a live table with millions of rows can require downtime, careful migration scripting, or both. Normalization is the discipline of structuring tables to avoid storing the same fact in more than one place, which prevents a whole category of data-integrity bugs where two copies of the same fact silently disagree. This guide covers the first three normal forms with concrete examples, walks through designing a normalized order-management schema from scratch, and covers when deliberately breaking normalization (denormalizing) is the right engineering tradeoff rather than a mistake.

1. Why Normalize? The Problem It Actually Solves

Without normalization, it's easy to design a table that repeats the same fact across many rows — for example, storing a customer's full name and email directly on every order row instead of in a separate customers table. The moment that customer's email changes, every order row needs to be updated, and if even one is missed, the database now contains two different 'true' answers to the same question — a state called an update anomaly. Normalization eliminates this by ensuring each fact is stored in exactly one place, referenced by foreign key wherever else it's needed.

2. First, Second, and Third Normal Form, Concretely

1NF requires that every column hold a single, atomic value — no comma-separated lists or repeating groups packed into one field (a 'tags' column holding 'red,blue,green' violates 1NF; it should be a separate table). 2NF (which only applies to tables with a composite primary key) requires that every non-key column depend on the entire primary key, not just part of it. 3NF requires that every non-key column depend only on the primary key, not on another non-key column — a customer's city stored on an orders table because it depends on the customer, not the order, violates 3NF and belongs in the customers table instead.

- 1NF: atomic values only — no comma-separated lists or repeating groups in a single column.

- 2NF: every non-key column depends on the whole primary key (relevant only for composite keys).
- 3NF: every non-key column depends only on the primary key, not on another non-key column (no 'transitive' dependencies).
- Most production schemas target 3NF as a practical default, deviating deliberately and knowingly when needed.

3. A Normalized Schema, End to End

Consider an order-management system: customers, products, orders, and the line items linking orders to products. Normalizing this correctly means customer details live only in the customers table, product details (including price) live only in the products table, and the order_items table exists purely to represent the many-to-many relationship between orders and products, carrying only the facts specific to that pairing (quantity, and the price at the time of purchase — deliberately duplicated because a product's price can change later, and a past order must preserve what was actually charged).

SQL

```
CREATE TABLE customers (  
    customer_id SERIAL PRIMARY KEY,  
    email VARCHAR(255) NOT NULL UNIQUE,  
    full_name VARCHAR(255) NOT NULL,  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE products (  
    product_id SERIAL PRIMARY KEY,  
    sku VARCHAR(50) NOT NULL UNIQUE,  
    name VARCHAR(255) NOT NULL,  
    unit_price DECIMAL(10, 2) NOT NULL  
);  
  
CREATE TABLE orders (  
    order_id SERIAL PRIMARY KEY,  
    customer_id INTEGER NOT NULL REFERENCES customers(customer_id),  
    status VARCHAR(20) NOT NULL DEFAULT 'pending',  
    placed_at TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE order_items (  
    order_item_id SERIAL PRIMARY KEY,  
    order_id INTEGER NOT NULL REFERENCES orders(order_id),  
    product_id INTEGER NOT NULL REFERENCES products(product_id),  
    quantity INTEGER NOT NULL CHECK (quantity > 0),  
    unit_price_at_purchase DECIMAL(10, 2) NOT NULL  
);
```

4. Primary Keys, Foreign Keys, and Referential Integrity

A primary key uniquely identifies each row in a table and, by definition, can never be null or duplicated. A foreign key is a column in one table that references a primary key in another, and its purpose is to let the database itself enforce that the reference is valid — you cannot insert an `order_items` row pointing at a `product_id` that doesn't exist in `products`, and (depending on the `ON DELETE` behavior configured) you cannot delete a product that's still referenced by existing `order_items` without explicitly deciding what should happen to those rows (restrict, cascade, or set null). Enforcing this in the database, rather than trusting application code to always check first, is what 'referential integrity' means in practice.

5. Choosing Correct Data Types

A schema's data types are a form of built-in validation, and choosing the wrong one is a common and costly mistake. Money should always be stored as `DECIMAL/NUMERIC` with a fixed number of decimal places, never `FLOAT` or `DOUBLE` — floating-point binary representation cannot exactly represent most decimal fractions, causing rounding errors that compound across many transactions. Dates and times should use dedicated `DATE/TIMESTAMP` types, not strings, so the database can validate, sort, and compare them correctly. Fixed-choice fields (order status, user role) benefit from an `ENUM` type or a foreign key to a lookup table rather than a free-text `VARCHAR` that allows any typo to slip through.

- Money: `DECIMAL(10,2)` or `NUMERIC`, never `FLOAT/DOUBLE` — avoids floating-point rounding errors.
- Dates/times: `DATE`, `TIME`, or `TIMESTAMP` types, not strings — enables correct sorting and range queries.
- Fixed sets of values: an `ENUM` type or a foreign key to a lookup table, not free-text `VARCHAR`.
- Always add `NOT NULL` to columns that should never be empty — don't rely on application code alone to enforce this.

6. When to Deliberately Denormalize

Normalization optimizes for data integrity and avoiding update anomalies, sometimes at the cost of requiring more joins to read data back out. For read-heavy workloads — dashboards, reporting, high-traffic public pages — the join cost can become a real performance problem, and deliberately denormalizing (storing a computed or duplicated value, like an order's total or a product's cached average rating, directly on a row instead of recalculating it via a join every time) is a legitimate, well-understood tradeoff. The key discipline is that denormalization should be a conscious decision made after normalizing correctly first, with a clear plan for how the duplicated value stays in sync (a trigger, an application-level update, or an async job), not a shortcut taken from the start to avoid learning normalization.

Common Pitfalls

- Storing money as FLOAT or DOUBLE instead of DECIMAL/NUMERIC, causing rounding errors that compound across transactions.
- Storing a comma-separated list of values in a single column (e.g. a 'tags' field) instead of a proper related table — violates 1NF and makes querying by individual value painful.
- Duplicating a fact (like a customer's email) across many rows instead of referencing it by foreign key, creating update anomalies when it changes.
- Not defining foreign key constraints, relying entirely on application code to keep references valid — orphaned rows eventually appear.
- Over-normalizing to the point where a single common query requires five or six joins, when a deliberate, documented denormalization would be the more practical choice.
- Missing NOT NULL and UNIQUE constraints at the database level, trusting application-level validation alone to prevent bad data.
- Forgetting to index foreign key columns, which are almost always used in JOIN conditions and WHERE clauses.

Key Takeaways

- Normalization prevents update anomalies by ensuring each fact is stored in exactly one place, referenced elsewhere by foreign key.
- 1NF: atomic column values. 2NF: non-key columns depend on the whole composite key. 3NF: non-key columns depend only on the primary key, not on each other.
- Foreign keys let the database itself enforce referential integrity, instead of trusting application code to never make a mistake.
- Use DECIMAL for money and proper DATE/TIMESTAMP types for dates — never FLOAT for currency or strings for dates.
- Denormalization is a legitimate, deliberate performance tradeoff for read-heavy workloads — but normalize first, then denormalize consciously, not by default.

Learn More

PostgreSQL Documentation — Data Types

<https://www.postgresql.org/docs/current/datatype.html>

GeeksforGeeks — Normal Forms in DBMS

<https://www.geeksforgeeks.org/normal-forms-in-dbms/>

PostgreSQL Documentation — Foreign Keys

<https://www.postgresql.org/docs/current/ddl-constraints.html#DDL-CONSTRAINTS-FK>